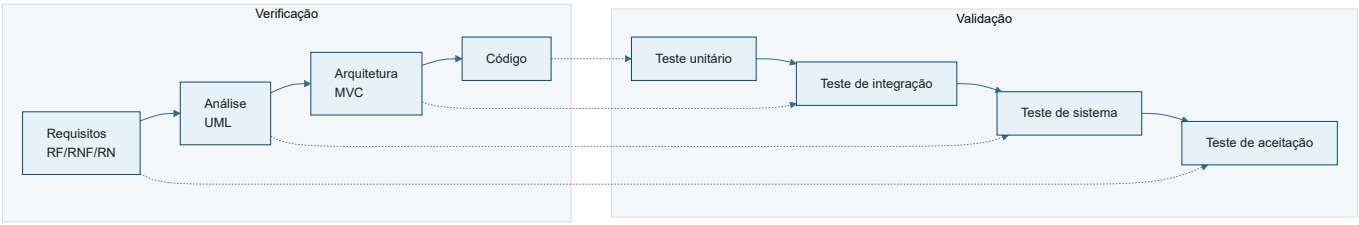
Plano de testes — Verificação e validação

1. Estratégia (Modelo V)



Nível do V	O que é verificado/validado	Como	Onde
Código ↔ Teste unitário	Funções e regras isoladas (conversões, CPF, placa, regras do Model, validação JS)	pytest e `node:test` automatizados	`tests/test_unitario_util.py`, `tests/test_unitario_models.py`, `tests/frontend/`
Arquitetura ↔ Teste de integração	Controller + Model + banco + templates juntos, via HTTP	`app.test_client()` do Flask com SQLite em memória	`tests/test_controllers.py`
Análise ↔ Teste de sistema	Fluxos completos dos casos de uso (UC04 → UC10 → UC11)	Cenários de ponta a ponta automatizados + roteiro manual	`tests/test_integracao.py` e seção 4
Requisitos ↔ Teste de aceitação	O sistema atende quem vai usar?	5 testadores externos com formulário padronizado	`03-formulario-teste-usuarios.md` e `laudos/`
Código ↔ Inspeção + regressão	O que o código **permite** e não deveria	Leitura dirigida de todos os módulos, com teste de regressão por achado	`06-revisao-tecnica-codigo.md`, `tests/test_revisao_tecnica.py`

Atividades de verificação (sem executar o sistema)

- Revisão do código contra os requisitos e as regras de negócio (tabela de rastreabilidade abaixo).
- Conferência de que o script `database/schema.sql` bate com os Models (gerado a partir do SQLAlchemy).
- Revisão dos diagramas UML contra as rotas e métodos implementados.
- **Inspeção dirigida de código** (11/11/2026): leitura de todos os módulos de `app/` com três perguntas fixas — *quem pode chamar isto?*, *o que acontece com entrada fora do previsto?*, *o que esta operação altera sem avisar?*. Resultado em `06-revisao-tecnica-codigo.md`.

Regra adotada para teste de regressão

Todo defeito encontrado ganha um teste **antes** da correção, e esse teste é conferido nos dois sentidos: ele precisa **falhar** no código com o defeito e passar no código corrigido. Um teste que passa nos dois casos está medindo o lugar errado e não serve como prova — foi o que aconteceu com dois dos testes escritos na revisão de 11/11, que precisaram ser reescritos.

2. Ambiente

Item	Configuração
SO	Windows 11
Python	3.11 · Flask 3.0.3 · SQLAlchemy 2.0.36 · pytest 8.3.3
Node.js	24 (somente testes de front-end)
Banco de teste	SQLite em memória, recriado a cada teste (isolamento total)
Navegadores (aceitação)	Chrome e celular Android/iOS

3. Casos de teste e rastreabilidade

CT	Descrição	Entrada	Resultado esperado	Requisito/Regra	Teste automatizado
CT01	Catálogo exibe veículos disponíveis com preço formatado	Veículo cadastrado	"R$ 72.900,00", "35.000 km"	RF01	`test_catalogo_lista_veiculo_com_preco_formatado`

CT02	Catálogo oculta vendidos	Veículo VENDIDO	Não aparece; detalhe dá 404	RN03	`test_catalogo_nao_exibe_vendidos`, `test_detalhe_inexistente_ou_vendido_retorna_404`
CT03	Filtros combinados	preço, combustível, câmbio, texto	Apenas os veículos que atendem	RF02	`test_catalogo_filtros_combinados`
CT04	Ordenação por preço	menor/maior preço	Ordem correta	RF03	`test_catalogo_ordenacao_por_preco`
CT05	Parâmetros inválidos na URL	`preco_min=abc&pagina=999`	Página carrega (200)	RNF07	`test_catalogo_parametros_invalidos_nao_quebram`
CT06	Placa mascarada na vitrine	Detalhe do veículo	"*-23", sem placa completa	RNF12	`test_detalhe_do_veiculo_oculta_placa`
CT07	Enviar proposta válida	Dados corretos	Proposta PENDENTE + mensagem de sucesso	RF05	`test_enviar_proposta_valida`
CT08	Proposta inválida	e-mail/telefone errados	HTTP 422, erros nos campos, dados mantidos	RF05, RNF07	`test_enviar_proposta_invalida_mostra_erros_e_mantem_dados`
CT09	Proposta para veículo vendido	Veículo VENDIDO	Recusada	RN03	`test_proposta_para_veiculo_vendido`
CT10	Login inválido	Senha errada	401 + mensagem genérica	RF06	`test_login_invalido`
CT11	Painel sem login	GET /admin/veiculos/	Redireciona ao login	RNF06	`test_painel_exige_login`
CT12	Open redirect	`?proximo=//site-malicioso.com`	Ignorado	RNF04	`test_login_ignora_redirecionamento_externo`
CT13	Cadastrar veículo com foto	PNG válido	Gravado, foto salva com nome UUID	RF07	`test_cadastrar_veiculo_com_foto`
CT14	Cadastro inválido	Placa "XX", preço 0	422 com mensagens	RN01, RN02	`test_cadastrar_veiculo_invalido_retorna_422`
CT15	Placa duplicada	Mesma placa	Erro no campo placa	RN02	`test_veiculo_placa_duplicada`
CT16	Placas antiga e Mercosul	ABC1234, abc-1d23	Aceitas e normalizadas	RN02	`test_veiculo_aceita_placas_antiga_e_mercosul`
CT17	Ano do modelo incoerente	fab. 2021, modelo 2023	Erro	RN04	`test_veiculo_ano_modelo_coerente`
CT18	Upload malicioso	PHP renomeado para .png; .exe	Rejeitado	RNF05	`test_cadastrar_veiculo_rejeita_arquivo_que_nao_e_imagem`, `..._rejeita_extensao`
CT19	Editar veículo	Novo preço/cor	Atualizado	RF08	`test_editar_veiculo`
CT20	Editar/excluir vendido	Veículo VENDIDO	Bloqueado	RN09	`test_editar_veiculo_vendido_bloqueado`, `test_veiculo_vendido_nao_pode_ser_editado_nem_excluido`
CT21	Vendedor tenta excluir	Perfil VENDEDOR	403	RF09, RNF06	`test_vendedor_nao_pode_excluir_veiculo`
CT22	Marca duplicada / em uso	"CHEVROLET", marca com carro	Erro / bloqueio	RN10, RF11	`test_marca_duplicada_mostra_erro`, `test_excluir_marca_em_uso`
CT23	Aceitar proposta	Proposta pendente	ACEITA + veículo RESERVADO	RN06	`test_aceitar_proposta_reserva_veiculo`
CT24	Segunda proposta aceita	Duas propostas	Segunda bloqueada	RN06	`test_somente_uma_proposta_aceita_por_veiculo`
CT25	Recusar proposta aceita	Proposta ACEITA	RECUSADA + veículo DISPONÍVEL	RN07	`test_recusar_proposta_aceita_libera_veiculo`

CT26	Registrar venda	Proposta aceita + CPF	VENDIDO, outras propostas recusadas	RN08	`test_venda_marca_veiculo_como_vendido_e_recusa_outras_propostas`
CT27	Vender duas vezes	Mesmo veículo	Bloqueado	RN09	`test_veiculo_nao_pode_ser_vendido_duas_vezes`
CT28	CPF inválido / de outra pessoa	CPF errado ou já usado	Erro	RN12	`test_venda_exige_cpf_valido`, `test_cliente_cpf_de_outra_pessoa`
CT29	CSRF	POST sem token	400 "Sua sessão expirou"	RNF04	`test_csrf_bloqueia_post_sem_token`
CT30	XSS	`<script>` no nome	Exibido escapado	RNF04	`test_campos_sao_escapados_contra_xss`
CT31	Senha com hash	Usuário criado	Hash ≠ senha	RNF03	`test_usuario_senha_e_armazenada_com_hash`
CT32	Fluxo completo	Cadastro → proposta → aceite → venda	Estado final consistente	UC04, UC06, UC10, UC11	`test_fluxo_completo_cadastro_proposta_venda`
CT33	Validação no navegador	Placa, e-mail, telefone, CPF, anos, valor	Mensagens corretas	RNF07	`tests/frontend/validacao.test.js`
CT34	Máscaras e moeda (JS)	Dígitos digitados	(11) 98765-4321 · 529.982.247-25 · R$ 85.000,00	RNF07	`tests/frontend/validacao.test.js`
CT35	Proposta pública não altera cadastro de terceiro	Proposta com e-mail de cliente existente	Nome e telefone originais preservados	RN13, RNF12	`test_cliente_existente_nao_e_reescrito_pela_vitrine`
CT36	Venda coerente com a proposta vinculada	Proposta de Maria, comprador João	Recusada com mensagem nomeando o autor da proposta	RN14	`test_c02_venda_recusa_proposta_de_outro_cliente`
CT37	Reserva respeitada na venda	Veículo RESERVADO, comprador diferente	Exige recusar a proposta aceita antes	RN06, RN14	`test_c03_veiculo_reservado_nao_e_vendido_a_terceiro_em_silencio`
CT38	Placa fora do formato no banco	Veículo com placa `ABC1`	Listagem do painel responde 200	RNF07	`test_c04_listagem_do_painel_responde_com_placa_fora_do_formato`
CT39	Limite de propostas	Duas propostas da mesma pessoa no mesmo veículo	Segunda recusada com aviso	RN15	`test_c05_uma_proposta_pendente_por_pessoa_e_veiculo`
CT40	Colisão de integridade na venda	Falha de unicidade no commit	Mensagem amigável, sem página 500	RNF07	`test_c06_colisao_de_integridade_na_venda_mostra_aviso`
CT41	Chave de sessão em produção	`AUTOVENDA_AMBIENTE=producao` sem `SECRET_KEY`	Aplicação recusa iniciar	RNF13	`test_s01_producao_recusa_a_chave_de_desenvolvimento`
CT42	Força bruta no login	5 senhas erradas seguidas	Bloqueio de 15 min, HTTP 429	RNF14	`test_s03_login_bloqueia_apos_tentativas_falhas`
CT43	Faixas invertidas no catálogo	`preco_min=200000&preco_max=10000`	Motivo explicado no campo, filtro descartado	RNF07	`test_u01_faixa_de_preco_invertida_explica_o_motivo`
CT44	Desempenho das listagens	3 e depois 9 propostas	Número de consultas não cresce com as linhas	RNF10	`test_d01_lista_de_propostas_nao_cresce_em_consultas`

4. Roteiro de teste manual de sistema (navegador)

> Executado em 11/11/2026 em Chromium dirigido por Playwright, contra a aplicação real. Todos os itens passaram; o M14 é parcial (atributo `aria-live` conferido, leitor de tela real não). Evidências e método em `07-sessao-teste-exploratorio.md`.

#	Passo	Resultado esperado	OK?
M01	Abrir o catálogo no celular	Layout em uma coluna, menu "Menu" abre/fecha	✓
M02	Digitar e-mail inválido no formulário de proposta e sair do campo	Mensagem vermelha imediata	✓
M03	Digitar telefone só com números	Máscara (11) 98765-4321 aplicada	✓
M04	Escolher foto acima de 5 MB no cadastro	Erro "A imagem deve ter no máximo 5 MB" antes do envio	✓ (arquivo de 6 MB)
M05	Escolher foto válida	Prévia aparece	✓
M06	Clicar duas vezes rápido em um botão de envio	Só um registro criado	✓ (verificado no envio de proposta; o bloqueio vale para todo formulário `data-validar`)
M07	Clicar em "Excluir" e cancelar a confirmação	Nada é excluído	✓
M08	Navegar só com Tab/Enter	Foco visível e todos os botões acessíveis	✓
M09	Acessar `/admin` em aba anônima	Redireciona ao login com aviso	✓
M10	Acessar `/veiculos/99999`	Página 404 amigável	✓
M11	Filtrar por marca e preço e depois digitar um termo na busca do topo	Os filtros continuam aplicados	✓
M12	Escolher foto acima do limite configurado	Mensagem cita o limite real, não um valor fixo	✓
M13	No celular, abrir o menu e pressionar Esc	Menu fecha e o foco volta ao botão "Menu"	✓
M14	Navegar o formulário de proposta com leitor de tela e errar o e-mail	O erro é anunciado ao sair do campo	Parcial

5. Critérios de aceite

- 100% dos testes automatizados passando.
- Cobertura do back-end ≥ 80%.
- Nenhum defeito de severidade **alta** aberto após a rodada com os 5 testadores.
- Todo defeito corrigido acompanhado de teste de regressão conferido nos dois sentidos (falha antes, passa depois).
- Todo defeito relatado registrado em `04-registro-de-correcoes.md`, corrigido ou justificado.

Relatório de testes automatizados

Data da execução inicial: 14/10/2026 **Reexecução de confirmação:** 11/11/2026 **Reexecução após a revisão técnica de código:** 11/11/2026 **Versão testada:** código atual da branch `main` do repositório **Evidências (saída completa):** `evidencias/resultado-pytest.txt` e `evidencias/resultado-frontend.txt`

1. Resumo

Suíte	Nível do Modelo V	Testes	Passaram	Falharam
`test_unitario_util.py`	Unitário	32	32	0
`test_unitario_models.py`	Unitário (regras de negócio)	56	56	0
`test_controllers.py`	Integração (HTTP + banco + templates)	39	39	0
`test_integracao.py`	Sistema (fluxos completos + comando seed)	3	3	0
`test_revisao_tecnica.py`	Regressão dos achados da revisão de código	28	28	0
`tests/frontend/validacao.test.js`	Unitário (front-end)	13	13	0
Total		**171**	**171**	**0**

Os 158 testes Python rodaram com `-W error::Warning`, ou seja, qualquer aviso seria tratado como falha.

Evolução da suíte

Marco	Testes Python	JavaScript	Total	Cobertura
Entrega inicial (14/10/2026)	128	13	141	93%
Após a revisão técnica de código (11/11/2026)	158	13	171	94%

Os 30 testes acrescentados são de **regressão**: cada um reproduz um defeito encontrado na inspeção de código documentada em `06-revisao-tecnica-codigo.md`. Todos foram conferidos nos dois sentidos — falham no código anterior à correção e passam no código atual. Um teste que nunca falhou não comprova nada, e essa conferência pegou dois testes fracos, que mediam o lugar errado e foram reescritos.

2. Cobertura de código do back-end

Módulo	Cobertura
`app/models/*` (regras de negócio)	94% a 100%
`app/controllers/*`	82% a 100%
`app/seguranca.py`	97%
`app/servicos/limites.py` (freio de força bruta)	95%
`app/servicos/armazenamento.py`	76% (sem cobertura: só os ramos de falha de disco ao remover arquivos)
Total	**94%** (meta: 80%)

3. Defeitos encontrados pelos testes durante o desenvolvimento

A suíte encontrou defeitos reais, que foram corrigidos antes da entrega:

#	Defeito	Como foi detectado	Correção
D01	No JavaScript, o ano do modelo **vazio** era convertido para `0` e mostrava a mensagem errada ("deve ser igual ao ano de fabricação…") em vez de "Informe o ano do modelo."	`validacao.test.js` › "ano do modelo igual ou um ano após a fabricação"	`validarAnoModelo` passou a tratar texto vazio antes de converter para número.
D02	Ao registrar a venda, o SQLAlchemy emitia aviso de objeto `Venda` fora da sessão durante o *autoflush*. Havia risco de a venda não ser associada ao cliente/vendedor em alguns cenários.	Execução com `-W error::Warning` em `test_venda_*`	`db.session.add(venda)` movido para antes de alterar o status das propostas.

4. Validação de segurança (resumo)

Verificação	Resultado
POST sem token CSRF	Bloqueado (400) ✓

POST com token CSRF válido	Aceito ✓
`<script>` em campo de texto	Exibido escapado ✓
Arquivo PHP com extensão `.png`	Rejeitado pela assinatura binária ✓
Redirecionamento externo após login	Ignorado ✓
Vendedor executando ação de admin	403 ✓
Cabeçalhos `X-Frame-Options` / `nosniff`	Presentes ✓
Força bruta no login (5 senhas erradas)	Bloqueado por 15 min (429) ✓
Senha correta durante o bloqueio	Também recusada ✓
Subir em produção com a chave de desenvolvimento	Aplicação recusa iniciar ✓
Proposta pública reescrevendo cadastro de terceiro	Bloqueado ✓
Venda vinculada à proposta de outro cliente	Bloqueado ✓

5. Teste manual em navegador (verificação feita pelo desenvolvedor)

Executado com a aplicação real (`flask run`, CSRF ligado, banco com `seed`):

Verificação	Resultado
Todas as rotas do painel respondem 200 após login	✓
Login sem token CSRF retorna 400	✓
Filtro "Híbrido" retorna apenas o Corolla Cross	✓
Validação em tempo real do e-mail no formulário de proposta	✓ mensagem exibida ao sair do campo
Máscara de telefone ao digitar `11987654321`	✓ `(11) 98765-4321`
Erros de JavaScript no console	Nenhum

> **Reexecutado em 11/11/2026.** A tabela acima foi preenchida na versão de 14/10/2026. Depois da revisão técnica de código, todo o roteiro manual (M01 a M14) foi executado em navegador real — Chromium dirigido por Playwright, com CSRF ligado e banco criado do zero. São **45 verificações sem nenhuma falha**, incluindo os itens novos M11 a M14. O relatório está em `07-sessao-teste-exploratorio.md` e as evidências em `materiais-locais/evidencias/sessao-exploratoria/`. O único item parcial é o M14: foi verificado que o elemento de erro tem `aria-live="polite"` e recebe o texto, mas não houve teste com leitor de tela real.

5.2 Sessão de teste exploratório em navegador

Os testes automatizados usam o cliente de teste do Flask: batem nas rotas e conferem o HTML, mas **não executam JavaScript, não têm layout e não têm teclado**. Máscara de campo, prévia de imagem, menu de celular, foco, botão Voltar e duplo clique ficam fora do alcance deles — justamente onde um testador humano tropeça primeiro.

Para cobrir essa faixa antes da rodada de aceitação, o roteiro que será entregue aos 5 testadores foi executado em navegador real pelo próprio autor, mais uma passada fora do roteiro (JavaScript desligado, entradas extremas, permissões, URLs maltratadas). Resultado: **45 verificações, nenhuma falha**. Detalhes, limites e as duas lições de método em `07-sessao-teste-exploratorio.md`.

Essa sessão **não substitui** os laudos dos testadores: ela percorre os caminhos que o autor imaginou, e o autor é quem escreveu o sistema.

5.1 Verificação por inspeção de código

Além da execução, foi feita uma leitura dirigida de todos os módulos de `app/`, procurando o que o sistema **permite** e não deveria — a classe de defeito que uma suíte de testes não encontra, porque testes só cobrem cenários que alguém imaginou. Resultado: 18 achados, 7 deles reproduzidos executando a aplicação, todos corrigidos. O relatório completo, com método e critério de gravidade, está em `06-revisao-tecnica-codigo.md`.

6. Como reproduzir

```
pip install -r requirements.txt
pytest -vv --cov=app
node --test "tests/frontend/*.test.js"
```

7. Teste de aceitação (usuários)

Está descrito em `03-formulario-teste-usuarios.md`. Os laudos dos 5 testadores ficam em `laudos/` e as correções em `04-registro-de-correcoes.md`.

Formulário de teste de aceitação (5 testadores)

Como conduzir

1. Escolha **5 pessoas**: colegas de curso ou profissionais da área.
2. Deixe o sistema acessível:

- **Opção A (mais simples):** publique em um serviço gratuito (Render, PythonAnywhere) e envie o link.
- **Opção B:** o testador clona o repositório e roda localmente seguindo o `README.md`.

1. Envie a cada testador:

- o link do sistema;
- os logins de demonstração (admin e vendedor);
- o **roteiro de tarefas** (seção 2);
- o **formulário** (seção 3), no Google Forms ou neste modelo em Word/PDF.

1. Peça prints das telas com erro.
2. Salve cada resposta em `laudos/` como `laudo-01-nome.pdf` … `laudo-05-nome.pdf`.

> **Importante:** o PDF da disciplina pede que as falhas **não sejam escondidas**. Registre tudo o que os testadores relatarem, inclusive o que você decidir não corrigir (com justificativa).

Roteiro de tarefas para o testador

Como visitante

1. Encontre um carro automático com preço até R$ 130.000.
2. Abra os detalhes de um carro e envie uma proposta. Tente antes enviar com um e-mail errado.

Como vendedor (`vendedor@autovenda.com` / `Vende@123`)

1. Cadastre um veículo com foto.
2. Tente cadastrar outro veículo com a **mesma placa**.
3. Aceite a proposta que você enviou na tarefa 2 e verifique o catálogo.
4. Registre a venda desse veículo, usando os **mesmos dados** que você informou na proposta (o sistema recusa vender a um comprador diferente do autor da proposta aceita).

Como administrador (`admin@autovenda.com` / `Admin@123`)

1. Cadastre uma marca nova e tente excluir uma marca que tenha carros.
2. Exclua o veículo cadastrado na tarefa 3.

Livre

1. Use o sistema pelo celular por alguns minutos.
2. Filtre por marca e faixa de preço e depois digite um termo na busca do topo: confira se os filtros continuam aplicados.
3. Tente enviar duas propostas seguidas para o mesmo carro com o mesmo e-mail: o sistema deve recusar a segunda, explicando o motivo.

Modelo do laudo de teste

Sistema: AutoVenda · **Versão/link testado:** _____

Campo	Resposta
Nome de quem testou	
Formação / área	☐ Colega de curso ☐ Profissional de TI ☐ Outro: ____
Data do teste	___/___/_____
Dispositivo e navegador	ex.: notebook + Chrome; celular Android + Chrome

1. O que testou e FUNCIONOU

Funcionalidade	Observação

2. O que testou e NÃO FUNCIONOU

Funcionalidade	O que aconteceu	O que deveria acontecer / sugestão de correção	Print (anexo nº)	Gravidade (Alta/Média/Baixa)

3. Funcionalidades NÃO TESTADAS

Funcionalidade	Motivo (não encontrou, faltou tempo, não existe no sistema, deu erro antes…)

4. Avaliação geral (1 = ruim, 5 = ótimo)

Critério	1	2	3	4	5
Facilidade de uso	☐	☐	☐	☐	☐
Clareza das mensagens de erro	☐	☐	☐	☐	☐
Aparência	☐	☐	☐	☐	☐
Uso no celular	☐	☐	☐	☐	☐
Velocidade	☐	☐	☐	☐	☐

5. Sugestões de melhoria

_____

6. Anexos (prints)

Anexo 1: _____ · Anexo 2: _____

Assinatura do testador: _____

Versão Google Forms

Crie o formulário com estas perguntas, na ordem:

1. Nome completo *(resposta curta, obrigatória)*
2. Formação / área *(múltipla escolha: Colega de curso, Profissional de TI, Outro)*
3. Data do teste *(data, obrigatória)*
4. Dispositivo e navegador usados *(resposta curta)*
5. Quais funcionalidades você testou e FUNCIONARAM? Descreva cada uma. *(parágrafo, obrigatória)*
6. Quais funcionalidades você testou e NÃO FUNCIONARAM? Descreva o que aconteceu e o que deveria acontecer. *(parágrafo)*
7. Gravidade do principal problema encontrado *(múltipla escolha: Alta (impede o uso), Média (atrapalha), Baixa (estético), Não encontrei problemas)*
8. Envie prints das telas com erro *(upload de arquivo, até 5 imagens)*
9. Quais funcionalidades você NÃO testou e por quê? *(parágrafo)*
10. Facilidade de uso *(escala 1–5)*
11. Clareza das mensagens *(escala 1–5)*
12. Aparência *(escala 1–5)*
13. Uso no celular *(escala 1–5)*
14. Sugestões de melhoria *(parágrafo)*

Depois de coletar as 5 respostas, use **Respostas › ⋮ › Imprimir todas as respostas › Salvar como PDF** para gerar o PDF exigido na entrega.

Registro de correções e melhorias pós-teste

> Preencha uma linha para **cada** item relatado nos laudos, inclusive os que não forem corrigidos. A coluna "Commit" liga a correção ao código (rastreabilidade requisito ↔ teste ↔ código).

Correções vindas dos testadores

#	Laudo	Problema/sugestão relatado	Gravidade	Decisão	O que foi feito	Commit	Teste que cobre
C01	laudo-01			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C02	laudo-02			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C03	laudo-03			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C04	laudo-04			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C05	laudo-05			☐ Corrigido ☐ Melhoria futura ☐ Não procede			

Correções encontradas pelos testes automatizados (antes da rodada com usuários)

#	Problema	Correção	Teste
D01	Ano do modelo vazio gerava mensagem errada no navegador	Tratamento de campo vazio em `validarAnoModelo`	`tests/frontend/validacao.test.js`
D02	Aviso do SQLAlchemy (Venda fora da sessão no autoflush)	`session.add(venda)` antes de atualizar as propostas	`test_venda_marca_veiculo_como_vendido_e_recusa_outras_propostas`

Correções vindas da revisão técnica de código (11/11/2026)

Inspeção de código feita antes de entregar a aplicação aos testadores, documentada em `06-revisao-tecnica-codigo.md`. São 18 achados, todos corrigidos; cada um tem teste de regressão que falha no código anterior.

#	Problema	Gravidade	O que foi feito	Teste que cobre
R01 (C-01)	Visitante reescrevia nome e telefone de cliente já cadastrado enviando proposta com o e-mail dele	Alta	`obter_ou_criar` só atualiza cadastro existente quando chamado pelo painel autenticado	`test_cliente_existente_nao_e_reescrito_pela_vitrine`
R02 (C-02)	Venda vinculada à proposta de um cliente e registrada no nome de outro	Alta	Conferência de que a proposta é do mesmo comprador	`test_c02_venda_recusa_proposta_de_outro_cliente`
R03 (S-01)	`SECRET_KEY` caía em silêncio para a chave de desenvolvimento publicada no repositório	Alta	Produção recusa subir sem chave própria	`test_s01_producao_recusa_a_chave_de_desenvolvimento`
R04 (C-03)	Vender veículo reservado a terceiro recusava a proposta aceita sem avisar ninguém	Média	Exige recusar a proposta antes, com mensagem nomeando o cliente da reserva	`test_c03_veiculo_reservado_nao_e_vendido_a_terceiro_em_silencio`
R05 (C-04)	Placa fora do formato derrubava a listagem do painel com `IndexError`	Média	Conferência de comprimento em `placa_formatada` + CHECK no `schema.sql`	`test_c04_listagem_do_painel_responde_com_placa_fora_do_formato`
R06 (C-05)	Propostas ilimitadas da mesma pessoa no mesmo veículo	Média	Uma proposta pendente por pessoa e por veículo	`test_c05_uma_proposta_pendente_por_pessoa_e_veiculo`
R07 (C-06)	Colisão de integridade no registro de venda virava página 500	Média	`IntegrityError` tratada com mensagem de aviso	`test_c06_colisao_de_integridade_na_venda_mostra_aviso`
R08 (U-01)	Faixa de preço invertida devolvia lista vazia sem explicação	Média	Validação no servidor, filtro descartado e motivo exibido no campo	`test_u01_faixa_de_preco_invertida_explica_o_motivo`
R09 (U-02)	A busca do catálogo descartava todos os filtros ativos	Média	Filtros viajam em campos ocultos no formulário de busca	`test_u02_busca_preserva_os_filtros_ativos`
R10 (S-02)	Cookie de sessão sem `Secure` e sessão sem prazo	Média	`SESSION_COOKIE_SECURE` em produção e validade de 8 horas	`test_s02_sessao_do_login_tem_prazo`

R11 (S-03)	Login sem freio contra força bruta	Média	5 falhas em 10 min bloqueiam por 15, resposta 429	`test_s03_login_bloqueia_apos_tentativas_falhas`
R12 (A-01)	Erros do JavaScript não eram anunciados por leitor de tela	Média	`aria-live="polite"` nos elementos de erro	`test_a01_erros_de_campo_tem_aria_live`
R13 (B-01)	`atualizado_em` mentia após `UPDATE` direto no PostgreSQL	Média	Trigger `tg_veiculo_atualizado_em`	Inspeção (sem instância PostgreSQL no ambiente)
R14 (U-03)	Faltava filtro de ano máximo no catálogo	Baixa	Campo "Ano até"	`test_u03_filtro_de_ano_maximo`
R15 (U-04)	Mensagem de tamanho de imagem fixa em "5 MB"	Baixa	Mensagem montada a partir do limite configurado	Roteiro manual M12
R16 (A-02)	Menu de celular não fechava com Esc nem devolvia o foco	Baixa	Esc e clique fora fecham o menu	Roteiro manual M13
R17 (B-02)	`schema.sql` sem CHECK de formato para placa, telefone e CPF	Baixa	CHECK acrescentados	Inspeção
R18 (D-01)	Consulta N+1 nas listas do painel	Baixa	Carregamento antecipado nas quatro listagens	`test_d01_lista_de_propostas_nao_cresce_em_consultas`

Observação metodológica. Um teste existente (`test_cliente_reaproveitado_pelo_email`) afirmava como correto o comportamento do defeito R01. Ele foi dividido em dois, separando o que é regra legítima (identificar o cliente pelo e-mail) do que era defeito (reescrever o cadastro). A suíte verde não apontava o problema porque ela própria o documentava como regra.

Resumo da rodada de aceitação

Indicador	Valor
Testadores	5
Itens relatados	
Corrigidos	
Adiados (melhoria futura)	
Não procedentes	
Nota média de facilidade de uso	

Boas práticas ao corrigir

1. Para cada bug, escreva primeiro um teste que reproduz o problema (ele deve falhar).
2. Corrija o código até o teste passar.
3. Rode a suíte inteira (`pytest` e `node --test`).
4. Faça um commit por correção, citando o item: `git commit -m "C03: mensagem de placa mais clara"`.
5. Atualize esta tabela e o vídeo final.

Revisão técnica do código — verificação por inspeção

Data: 11/11/2026 **Versão revisada:** branch `main`, commit `715a494` **Natureza:** atividade de **verificação** do Modelo V (inspeção de código), complementar aos testes automatizados e anterior à rodada de validação com os 5 testadores.

> Esta revisão **não substitui** os laudos dos testadores previstos na Situação 3. Ela é uma inspeção técnica do próprio código, feita antes de colocar a aplicação na mão dos avaliadores, justamente para que eles não gastem a rodada de testes relatando defeitos que já eram detectáveis por leitura.

---

1. Por que fazer esta revisão

O plano de testes já previa, entre as atividades de verificação, a "revisão do código contra os requisitos e as regras de negócio". Os 141 testes automatizados existentes verificavam o que o sistema **deve** fazer. Esta revisão foi atrás do outro lado: o que o sistema **permite** fazer e não deveria.

Essa distinção importa porque uma suíte de testes só cobre os cenários que alguém imaginou. Um teste que afirma "a proposta é criada com os dados do cliente" passa tanto no sistema correto quanto no sistema que deixa qualquer visitante reescrever o cadastro de outra pessoa — porque ninguém escreveu o teste do segundo caso. Ler o código procurando o que ele não proíbe encontra uma classe de defeito que a suíte, por construção, não encontraria.

2. Método

Etapa	O que foi feito
Leitura dirigida	Todos os módulos de `app/` lidos linha a linha, com três perguntas fixas: *quem pode chamar isto?*, *o que acontece com entrada fora do previsto?* e *o que esta operação altera sem avisar?*
Reprodução	Cada suspeita foi transformada em um roteiro executável contra a aplicação real, com banco em memória. Só os achados que de fato reproduziram viraram item da tabela.
Classificação	Gravidade pelo impacto no negócio e na confiança do dado, não pela dificuldade da correção.
Correção com teste	Para cada achado corrigido, um teste de regressão que **falha** no código antigo e passa no novo.

Critério de gravidade

Gravidade	Significado
Alta	Dado de terceiro é corrompido, histórico fica inconsistente, ou controle de acesso é contornável.
Média	Defeito reproduzível que prejudica o uso ou abre risco operacional, com contorno possível.
Baixa	Divergência, incômodo ou desperdício sem perda de dado.

3. Achados

Identificadores: **C** correção/regra de negócio · **S** segurança · **U** usabilidade e validação · **A** acessibilidade · **B** banco de dados · **D** desempenho.

ID	Gravidade	Achado	Como foi comprovado	Situação
C-01	**Alta**	Qualquer visitante reescreve o nome e o telefone de um cliente já cadastrado, bastando enviar uma proposta usando o e-mail dele. `Cliente.obter_ou_criar` gravava os dados recebidos por cima do cadastro existente.	Reproduzido: cadastro "Maria Souza / 11987654321" virou "Invasor Qualquer / 11000000000" após uma proposta pública.	Corrigido
C-02	**Alta**	Uma venda podia ser vinculada à proposta de uma pessoa e registrada no nome de outra. O histórico passava a afirmar duas coisas diferentes sobre o mesmo negócio.	Reproduzido: venda com `proposta.cliente = Maria Souza` e `venda.cliente = Joao Outro`.	Corrigido
S-01	**Alta**	`SECRET_KEY` caía em silêncio para a chave de desenvolvimento, que está publicada no repositório. Quem a lesse conseguiria assinar cookies de sessão válidos e entrar no painel como qualquer usuário.	Inspeção de `app/config.py`.	Corrigido
C-03	Média	Registrar a venda de um veículo **reservado** para um terceiro rebaixava a proposta aceita para RECUSADA, sem nenhum aviso ao vendedor. O cliente que teve a proposta aceita perdia a reserva em silêncio.	Reproduzido: proposta de Carlos (ACEITA) virou RECUSADA ao vender para Pedro.	Corrigido
C-04	Média	`placa_formatada` estourava `IndexError` com placa de menos de 5 caracteres, derrubando a listagem inteira do painel. Alcançável por carga feita direto no banco, que não tinha CHECK de formato.	Reproduzido: `IndexError: string index out of range` com placa `ABC1`.	Corrigido
C-05	Média	O formulário público aceitava propostas ilimitadas da mesma pessoa no mesmo veículo. A fila do vendedor enchia de duplicatas e a tabela ficava exposta a inundação automatizada.	Reproduzido: 22 propostas no mesmo veículo em um laço, sem qualquer recusa.	Corrigido

C-06	Média	`IntegrityError` ao registrar venda não era tratada e virava página de erro 500. Dois envios simultâneos do formulário passam pela validação e só colidem na restrição `uq_venda_veiculo`. O controller de veículos já tratava o caso; o de vendas, não.	Inspeção comparativa dos dois controllers.	Corrigido
U-01	Média	Faixa de preço invertida pela URL devolvia "0 veículos encontrados" sem explicar nada. O JavaScript avisava, o servidor não — quem chegasse por link antigo ou com JS desligado só via a lista vazia.	Reproduzido: `/?preco_min=200000&preco_max=10000` → 200, zero resultados, nenhuma mensagem.	Corrigido
U-02	Média	A busca do topo do catálogo descartava todos os filtros ativos. Quem filtrava marca e faixa de preço e depois digitava um termo perdia a seleção inteira sem aviso.	Inspeção de `catalogo/index.html`: o formulário de busca não carregava os filtros.	Corrigido
S-02	Média	Cookie de sessão sem `Secure` e sessão sem prazo de validade: uma sessão esquecida em máquina compartilhada permanecia válida indefinidamente.	Inspeção de `app/config.py`.	Corrigido
S-03	Média	Login sem qualquer freio contra força bruta: tentativas ilimitadas de senha, sem atraso nem bloqueio.	Inspeção de `auth_controller.py`.	Corrigido
A-01	Média	As mensagens de erro inseridas pelo JavaScript ficavam em elementos sem `aria-live`. Quem usa leitor de tela não era avisado do erro de validação e só percebia o problema ao tentar enviar.	Inspeção de `_macros.html`.	Corrigido
B-01	Média	`atualizado_em` só era mantido pela aplicação (`onupdate` do SQLAlchemy). Um `UPDATE` feito direto no PostgreSQL deixava a coluna mentindo sobre a última alteração.	Inspeção de `database/schema.sql`.	Corrigido
U-03	Baixa	Não havia filtro de ano máximo, só "ano a partir de". Quem procurava carro mais antigo por preço não tinha como limitar.	Inspeção do catálogo.	Corrigido
U-04	Baixa	A mensagem de tamanho de imagem dizia "5 MB" fixo, ignorando o limite real configurado. Mudar a configuração faria a tela mentir.	Inspeção de `app.js`.	Corrigido
A-02	Baixa	O menu de celular não fechava com Esc nem devolvia o foco ao botão: quem navega por teclado precisava percorrer todos os links até o fim para sair.	Inspeção de `app.js`.	Corrigido
B-02	Baixa	`schema.sql` não tinha CHECK de formato para placa, telefone e CPF, embora o Model validasse os três. Carga por SQL gravava dado quebrado — origem do C-04.	Inspeção comparativa Model × schema.	Corrigido
D-01	Baixa	Consulta N+1 nas listas do painel: cada linha de proposta, venda ou veículo disparava consultas próprias para cliente, veículo e marca.	Medido por contagem de consultas: a lista de propostas passou de 10 para 22 consultas ao triplicar as linhas.	Corrigido

3.1 Um achado que não se confirmou

A suspeita inicial incluía N+1 **no catálogo público**. A medição mostrou que ele não existe: o controller já carrega todas as marcas para montar o filtro lateral, e o acesso a `veiculo.marca` em cada cartão é respondido pelo mapa de identidade do SQLAlchemy, sem ir ao banco.

O registro fica aqui de propósito. O primeiro teste escrito para esse caso **passava com e sem a correção** — ele media o lugar errado. Um teste que nunca falhou não prova nada, e foi reescrito para medir a listagem do painel, onde o defeito é real e a contagem de fato sobe de 4 para 8 consultas. O carregamento antecipado no catálogo foi mantido como proteção, mas não é contado como correção de defeito.

4. O que foi corrigido

ID	Correção aplicada	Arquivo
C-01	`obter_ou_criar` ganhou o parâmetro `atualizar_cadastro`. A vitrine pública passa `False` e nunca reescreve cadastro existente; o painel passa `True`, porque ali quem digita é funcionário autenticado conferindo os dados com o comprador.	`app/models/cliente.py`
C-02	A venda confere se a proposta vinculada é do mesmo comprador e, se não for, recusa com mensagem nomeando o dono da proposta.	`app/models/venda.py`
C-03	Vender veículo reservado para terceiro passou a exigir que a proposta aceita seja recusada antes, com mensagem explicando para quem o veículo está reservado.	`app/models/venda.py`
C-04	`placa_formatada` confere o comprimento antes de indexar e devolve o valor bruto quando ele está fora do formato.	`app/models/veiculo.py`
C-05	Uma proposta pendente por pessoa e por veículo. Depois de recusada, a pessoa pode propor de novo.	`app/models/proposta.py`
C-06	`IntegrityError` tratada no registro de venda, com mensagem de aviso e redirecionamento para a lista.	`app/controllers/venda_controller.py`
S-01	`AUTOVENDA_AMBIENTE=producao` faz a aplicação **recusar subir** com a chave de desenvolvimento. Fora de produção, registra aviso no log.	`app/__init__.py`, `app/config.py`
S-02	`SESSION_COOKIE_SECURE` ligado em produção e sessão com validade de 8 horas.	`app/config.py`, `auth_controller.py`
S-03	Freio de força bruta por e-mail + origem da requisição: 5 falhas em 10 minutos bloqueiam por 15, resposta HTTP 429. A chave combina os dois dados de propósito — só o e-mail deixaria um atacante bloquear a conta de um vendedor por maldade; só o IP deixaria passar quem varre vários e-mails.	`app/servicos/limites.py`, `auth_controller.py`
U-01	Faixas invertidas de preço e de ano são validadas no servidor, descartadas da consulta e explicadas no próprio campo.	`app/models/veiculo.py`, `catalogo_controller.py`

U-02	O formulário de busca leva os filtros ativos em campos ocultos.	`catalogo/index.html`
U-03	Filtro "Ano até" no catálogo.	`app/models/veiculo.py`, `catalogo/index.html`
U-04	A mensagem de tamanho é montada a partir do limite configurado.	`app.js`
A-01	`aria-live="polite"` nos elementos de erro de campo.	`_macros.html`, `auth/login.html`
A-02	Esc fecha o menu e devolve o foco; clique fora também fecha.	`app.js`
B-01	Trigger `tg_veiculo_atualizado_em` mantém a coluna no PostgreSQL, independentemente de quem fez o `UPDATE`.	`database/schema.sql`
B-02	CHECK de formato para placa, telefone e CPF.	`database/schema.sql`
D-01	Carregamento antecipado (`joinedload` / `contains_eager`) nas listas de propostas, vendas, veículos e no painel.	controllers e `app/models/veiculo.py`

5. Rastreabilidade crítica → teste

Os testes ficam em `tests/test_revisao_tecnica.py`, nomeados com o identificador do achado. Cada um foi conferido nos dois sentidos: **falha** no código anterior à correção e passa no código atual.

ID	Teste de regressão
C-01	`test_cliente_existente_nao_e_reescrito_pela_vitrine`, `test_cliente_existente_e_atualizado_pelo_painel` (em `test_unitario_models.py`)
C-02	`test_c02_venda_recusa_proposta_de_outro_cliente`, `test_c02_venda_aceita_proposta_do_proprio_comprador`
C-03	`test_c03_veiculo_reservado_nao_e_vendido_a_terceiro_em_silencio`, `test_c03_apos_recusar_a_reserva_a_venda_a_terceiro_e_liberada`
C-04	`test_c04_placa_curta_nao_quebra_a_formatacao`, `test_c04_listagem_do_painel_responde_com_placa_fora_do_formato`
C-05	`test_c05_uma_proposta_pendente_por_pessoa_e_veiculo`, `test_c05_nova_proposta_liberada_depois_de_recusada`, `test_c05_proposta_duplicada_exibe_aviso_na_vitrine`, `test_c05_pessoas_diferentes_propoem_no_mesmo_veiculo`
C-06	`test_c06_colisao_de_integridade_na_venda_mostra_aviso`
S-01	`test_s01_producao_recusa_a_chave_de_desenvolvimento`, `test_s01_producao_sobe_com_chave_propria`
S-02	`test_s02_cookie_de_sessao_tem_protecoes`, `test_s02_sessao_do_login_tem_prazo`
S-03	`test_s03_login_bloqueia_apos_tentativas_falhas`, `test_s03_login_correto_zera_o_contador`, `test_controle_de_tentativas_libera_depois_do_prazo`, `test_controle_de_tentativas_separa_as_chaves`
U-01	`test_u01_faixa_de_preco_invertida_explica_o_motivo`, `test_u01_faixa_de_ano_invertida_explica_o_motivo`, `test_u01_faixa_valida_continua_filtrando`
U-02	`test_u02_busca_preserva_os_filtros_ativos`
U-03	`test_u03_filtro_de_ano_maximo`
A-01	`test_a01_erros_de_campo_tem_aria_live`
D-01	`test_d01_lista_de_propostas_nao_cresce_em_consultas`, `test_d01_lista_de_veiculos_do_painel_nao_consulta_marca_por_linha`
U-04, A-02	Comportamento de DOM, verificado pelo roteiro manual (M11 a M14 do plano de testes)
B-01, B-02	Script SQL; conferido por inspeção, sem instância PostgreSQL neste ambiente

6. Efeito na suíte

	Antes	Depois
Testes Python	128	158
Testes JavaScript	13	13
Total	**141**	**171**
Cobertura do back-end	93%	94%

Um teste **antigo precisou ser reescrito**: `test_cliente_reaproveitado_pelo_email` afirmava que uma segunda chamada a `obter_ou_criar` atualizava o telefone do cliente — ou seja, o teste descrevia o comportamento do defeito C-01 como se fosse o comportamento desejado. Ele foi dividido em dois: um que verifica a identificação pelo e-mail (que continua correta) e outro que verifica que o cadastro não é reescrito pela via pública.

Vale registrar o que esse caso ensina: a suíte não apontava o defeito porque ela própria o documentava como regra. Teste verde não é prova de código correto — é prova de que o código faz o que o autor do teste esperava.

7. O que ficou de fora

Item	Por quê
Freio de força bruta distribuído	O controle atual guarda as tentativas em memória do processo, o que basta para a loja rodar em um processo só. Com vários processos, a contagem precisa ir para armazenamento compartilhado. A classe foi escrita com essa troca em mente: só a implementação muda.
Confirmação do e-mail do proponente	Hoje nada garante que quem envia a proposta é dono do e-mail informado. A correção do C-01 fecha o estrago (ninguém reescreve cadastro alheio), mas uma proposta ainda pode ser enviada com e-mail de terceiro. Resolver exige envio de e-mail de confirmação, fora do escopo desta entrega.
Verificação dos CHECK e da trigger no PostgreSQL	O ambiente de desenvolvimento roda SQLite, que não aceita a sintaxe `~` de expressão regular nem `CREATE TRIGGER` em PL/pgSQL. As adições ao `schema.sql` foram conferidas por inspeção e precisam de execução em uma instância PostgreSQL antes de ir para produção.
Paginação do catálogo com muitos registros	Com o volume de demonstração (9 veículos) não há como medir. Fica como ponto de atenção para a rodada de validação.

8. Como o resultado entra na Situação 3

1. Os defeitos corrigidos aqui estão registrados em `04-registro-de-correcoes.md`, na seção de correções anteriores à rodada com usuários.
2. Os casos de teste novos entraram no `01-plano-de-testes.md` (CT35 a CT44 e M11 a M14).
3. Os números atualizados estão no `02-relatorio-de-testes.md`.
4. Os 5 testadores recebem a versão já corrigida, o que libera a rodada de aceitação para encontrar o que só o uso real revela.

Sessão de teste exploratório em navegador

Data: 11/11/2026 **Versão testada:** `main` após as correções da revisão técnica de código **Ambiente:** Chromium headless dirigido por Playwright 1.62, contra a aplicação real (`flask run`, CSRF ligado, banco criado do zero com `flask seed`) **Evidências:** 13 capturas de tela e 2 arquivos de resultado em `materiais-locais/evidencias/sessao-exploratoria/`

> **O que esta sessão não é.** Ela **não substitui** os laudos dos 5 testadores exigidos pela Situação 3, e nada aqui foi escrito em nome de outra pessoa. É o próprio autor executando, em navegador, o mesmo roteiro que será entregue aos testadores — para que eles não gastem a rodada relatando defeitos que uma execução cuidadosa já encontraria.

---

1. Por que fazer antes da rodada com usuários

Os testes automatizados usam o cliente de teste do Flask: eles batem nas rotas e conferem o HTML devolvido, mas **não executam JavaScript, não têm layout e não têm teclado**. Tudo que depende de navegador — máscara de campo, prévia de imagem, menu de celular, foco, botão Voltar, duplo clique — fica fora do alcance deles. É exatamente a faixa onde um testador humano tropeça primeiro.

Esta sessão cobre essa faixa com um navegador de verdade.

2. O que foi executado

Passada	Escopo	Verificações
1	As 11 tarefas do roteiro que será entregue aos testadores (`03-formulario-teste-usuarios.md`), como visitante, vendedor e administrador	23
2	O que um testador curioso faz fora do roteiro: teclado, JavaScript desligado, botão Voltar, entradas extremas, permissões e URLs maltratadas	22
	Total	**45**

Resultado: 45 verificações, nenhuma falha. A única marcação de atenção é um registro `422` no console do navegador, produzido de propósito pela tentativa de cadastrar placa duplicada — é a resposta correta do servidor, não um erro de JavaScript.

Passada 1 — roteiro dos testadores

ID	Verificação	Resultado
T01	Filtrar automático até R$ 130.000	3 veículos, nenhum acima do teto
T02a	E-mail inválido avisa ao sair do campo	"Informe um e-mail válido (ex.: nome@dominio.com)."
T02b	Proposta válida é aceita	Mensagem de sucesso exibida
T03a–d	Login do vendedor, prévia da foto, placa em maiúsculas, cadastro gravado	Todos conforme esperado
T04	Placa duplicada	"A placa EXP1A11 já está cadastrada em outro veículo."
T05a–b	Aceitar proposta reserva o veículo e o catálogo avisa	Conforme esperado
T06a–c	Venda pré-preenchida pela proposta, registrada, veículo sai do catálogo	Conforme esperado
T07a–b	Admin cadastra marca; marca com veículos não oferece exclusão	Conforme esperado
T08	Admin exclui o veículo cadastrado	Conforme esperado
T09a	Catálogo no celular (390 px)	Documento com 390 px, sem rolagem horizontal
T10	Busca do topo preserva os filtros	`?preco_max=130000&cambio=AUTOMATICO&ordem=recentes&q=Corolla`
T11	Segunda proposta do mesmo e-mail	Recusada com explicação

Passada 2 — fora do roteiro

ID	Verificação	Resultado
H-*	Higiene de HTML em catálogo, detalhe, login e página 404: id duplicado, imagem sem `alt`, campo sem rótulo, quantidade de `h1`	Nenhum problema nas 4 páginas
M08	Foco visível ao navegar por Tab	11 paradas, todas com contorno visível; ordem lógica começando pelo "Pular para o conteúdo"
X01	**Validação com JavaScript desligado**	O servidor devolveu os erros nos campos; a aplicação não depende do JavaScript para validar
X02	Formulário utilizável após o botão Voltar	Botão de envio volta habilitado

M06	Duplo clique no envio	Botão desabilitado dentro do próprio `submit`; 3 cliques em rajada geraram **1** requisição
X03	Nome com 300 caracteres e valor de R$ 999.999.999.999	Erros de validação exibidos
X04	Emoji, acentuação e `<script>` na mensagem	Proposta aceita e gravada corretamente
X05	**XSS armazenado**	O `<script>` digitado pelo visitante aparece escapado no painel do vendedor
X06	Emoji sobrevive à ida e volta do banco	🚗 preservado
X07	Vendedor tentando criar marca por requisição direta	HTTP 400
X08	Painel esconde do vendedor o formulário de marcas	Apenas o aviso é exibido
M03	Máscara de telefone ao digitar só números	`(11) 98765-4321`
M07	Cancelar a confirmação de exclusão	11 marcas antes, 11 depois
M09	`/admin/veiculos/` em aba anônima	Redireciona para `/login?proximo=/admin/veiculos/` com aviso
U-*	URLs maltratadas: `pagina=99999`, `preco_min=abc`, `marca=99999`, `/veiculos/0`, `/admin/veiculos/abc/editar`	200 ou 404, nenhuma página de erro 500

3. Roteiro manual do plano de testes

Os itens M01 a M14 de `01-plano-de-testes.md` foram executados nesta sessão:

Item	Situação	Onde está a evidência
M01 Layout no celular	✓	T09a · `t09-celular.png`
M02 E-mail inválido	✓	T02a · `t02-erro-email.png`
M03 Máscara de telefone	✓	M03
M04 Foto acima do limite	✓	M12 (arquivo de 6 MB)
M05 Prévia da foto	✓	T03b · `t03-cadastro-preenchido.png`
M06 Duplo clique	✓	M06
M07 Cancelar exclusão	✓	M07
M08 Navegação por teclado	✓	M08
M09 Painel em aba anônima	✓	M09
M10 Endereço inexistente	✓	H-erro 404 e `U-/veiculos/0`
M11 Filtros preservados na busca	✓	T10 · `t10-busca-com-filtros.png`
M12 Mensagem de tamanho da imagem	✓	M12
M13 Menu fecha com Esc	✓	M13 (fechou e devolveu o foco ao botão)
M14 Erro anunciável por leitor de tela	**Parcial**	Verificado que o elemento tem `aria-live="polite"` e recebe o texto do erro. **Não foi verificado com um leitor de tela real** (NVDA ou VoiceOver); isso continua valendo a pena pedir a um testador que use leitor de tela.

4. Duas lições de método

Nenhum defeito da aplicação apareceu, mas a sessão produziu dois **falsos positivos** que merecem registro, porque mostram como um teste mal medido engana:

O foco por teclado "invisível". A primeira execução acusou que o primeiro elemento tabulado não tinha contorno de foco. Investigando, o elemento era o `body`: o foco ainda não havia entrado no documento quando a medição começou. Com o foco posicionado antes do primeiro `Tab`, as 11 paradas aparecem corretamente, todas com contorno. O defeito era do script, não da página.

O botão que "não desabilitava". A primeira execução acusou que o botão de envio seguia clicável depois do clique. O que acontecia é que a leitura ocorria **depois** da navegação, já na página seguinte, onde o botão é naturalmente novo e habilitado. Medindo dentro do próprio evento `submit` — antes de a navegação destruir a página — o botão está desabilitado, e três cliques em rajada geram uma única requisição.

Os dois casos repetem a lição já registrada na revisão de código: **um resultado de teste só vale depois de você entender por que ele deu aquilo**. Os scripts foram corrigidos para não repetir os dois enganos.

5. Limite honesto deste resultado

Quarenta e cinco verificações sem falha **não significam que o sistema está correto**. Significam que ele se comporta como eu esperava nos caminhos que eu pensei em percorrer — e eu sou a pessoa que escreveu o sistema, logo meus caminhos são justamente os que ele já atende.

O que um testador traz e esta sessão não tem como produzir:

- expectativas diferentes das do autor ("eu esperava que o filtro limpasse sozinho");
- julgamento sobre clareza de texto, rótulo e mensagem de erro;
- uso em aparelho, navegador e conexão reais, não em Chromium headless;
- as tarefas que a pessoa tenta fazer e que ninguém previu no roteiro.

Por isso a rodada com os 5 testadores continua pendente e continua sendo a parte que decide a Situação 3.

6. Como repetir a sessão

```
 # Terminal 1 — aplicação com banco limpo
flask --app run seed
flask --app run run --port 5099

# Terminal 2
npm install --no-save playwright@1.62.1
node materiais-locais/sessao-teste-exploratorio.js materiais-locais/evidencias/sessao-exploratoria
node materiais-locais/sessao-teste-exploratorio-2.js materiais-locais/evidencias/sessao-exploratoria
```

Os scripts geram um identificador por rodada nos e-mails usados, então podem ser executados várias vezes sobre o mesmo banco sem esbarrar na regra RN15 (uma proposta pendente por pessoa e veículo).